

Database Connection & Security Guide

How this application connects to a SQL database, how to point it at your own database, and the security practices built into the connection layer — including database-driven CORS.

1. How the Connection Works

Every patient record served by the dashboard is read from a single SQL table, `patient_visits`, via SQLAlchemy — Python's standard database toolkit. SQLAlchemy is what makes the database itself swappable: the same application code works unchanged against SQLite, PostgreSQL, or MySQL, because SQLAlchemy abstracts the differences between them behind one connection URL.

The connection is configured entirely through environment variables in `backend/.env` — never hardcoded in source code. This file is the single source of truth for credentials, and it is excluded from version control (`.gitignore`) so secrets never end up in a git history.

Where each piece lives

Field	Description
<code>backend/.env</code>	Your real credentials. Never committed to git.
<code>backend/.env.example</code>	A template with placeholder values — safe to commit, shows every setting available.
<code>app/config/settings.py</code>	Loads <code>.env</code> into a typed <code>Settings</code> object and resolves the final connection URL.
<code>app/db/session.py</code>	Builds the SQLAlchemy engine/connection pool from that URL and manages sessions.
<code>app/db/models.py</code>	Defines the <code>patient_visits</code> and <code>allowed_origins</code> table structures (the ORM models).
<code>app/services/data_service.py</code>	The only place that queries <code>patient_visits</code> — every route/analytics function reads from here.
<code>app/services/cors_service.py</code>	Caches the <code>allowed_origins</code> table in memory and refreshes it periodically.
<code>app/middleware/dynamic_cors.py</code>	CORS middleware that checks incoming request origins against that cache instead of a static list.
<code>scripts/seed_dummy_data.py</code>	Creates the schema, inserts synthetic demo data, and ensures default CORS origins exist.

2. Connecting Your Own Database

Open backend/.env and set **either** a single connection string, **or** the individual DB_* fields — not both. If DATABASE_URL is set, it always takes priority.

Option A — PostgreSQL (recommended for production)

```
DATABASE_URL=postgresql+psycopg2://patient360_user:YOUR_PASSWORD@your-host:5432/patient360_db
DB_SSL_MODE=require
```

Option B — MySQL

```
DATABASE_URL=mysql+pymysql://patient360_user:YOUR_PASSWORD@your-host:3306/patient360_db
```

Option C — SQLite (zero setup, ships with a ready-to-use dummy database)

```
DATABASE_URL=sqlite:///./patient360_dummy.db
```

SQLite needs no server, no username, and no password — it's a single file (patient360_dummy.db, included in this project, pre-populated with 550 synthetic records across 80 patients). This is the fastest way to run the app locally; switch to PostgreSQL/MySQL for anything beyond local development.

Option D — individual fields instead of one URL

Useful when your secrets are injected separately (e.g. Kubernetes secrets, some CI systems). Leave DATABASE_URL empty and set:

```
DB_DRIVER=postgresql+psycopg2
DB_HOST=your-host
DB_PORT=5432
DB_NAME=patient360_db
DB_USER=patient360_user
DB_PASSWORD=YOUR_PASSWORD
```

After changing .env

- Restart the backend (uvicorn / docker compose restart backend).
- The app automatically creates the patient_visits and allowed_origins tables if they don't exist yet — this is safe and idempotent; it never touches existing data.
- To load demo data into a freshly connected database, run: python -m scripts.seed_dummy_data (from the backend/ directory).
- Check the connection any time from the app itself: the “Database” page in the dashboard, or GET /api/database/status.

3. Security Practices

The goal is straightforward: only this application, using its own credentials, can read the database — and nothing in the request path lets a user manipulate the SQL being run, or call the API from an origin nobody approved.

Credentials never leave .env

- Database username/password/connection string live only in backend/.env, which is git-ignored — .env.example (placeholders only) is what gets committed.
- The connection-status endpoint and dashboard page report driver, host, and record counts, but deliberately never echo the username or password back to the browser.
- DB_ECHO_SQL defaults to false — enabling it logs every query (and its parameter values), so it should only ever be turned on for local debugging, never in production.

SQL injection is prevented structurally

Every query in this app is built with SQLAlchemy's query construction (`select(...)`, ORM filters, bulk inserts) rather than string-formatted SQL. Values are always sent to the database as bound parameters, not concatenated into the query text — this is what actually prevents SQL injection, regardless of what a patient name or search term contains.

Use a least-privilege database user

Don't connect with a database superuser. Create a dedicated account scoped to only what this app needs:

```
-- PostgreSQL example
CREATE USER patient360_user WITH PASSWORD 'a-strong-generated-password';
CREATE DATABASE patient360_db OWNER patient360_user;
GRANT CONNECT ON DATABASE patient360_db TO patient360_user;
GRANT USAGE, CREATE ON SCHEMA public TO patient360_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO patient360_user;
```

For a read-only analytics deployment, drop INSERT/UPDATE/DELETE entirely and grant only SELECT — the dashboard itself never writes to `patient_visits`; only the seed script does.

Encrypt the connection (SSL/TLS)

For any database that isn't on localhost, set `DB_SSL_MODE` to require or stricter (`verify-ca` / `verify-full` for certificate-pinned connections). SQLite has no network connection to secure, so this setting doesn't apply to it.

Network-level access control

- Restrict the database's firewall/security-group rules to only the backend's IP range — a database should never be reachable from the open internet.
- In `docker-compose.yml`, the Postgres service is only exposed to other containers on the compose network by default; the host port mapping is there for local debugging and can be removed entirely for production.
- Rotate `DB_PASSWORD` periodically and after any suspected exposure — since it's read purely from `.env`, rotating it is a one-line change plus a backend restart.

Connection pool limits

DB_POOL_SIZE, DB_MAX_OVERFLOW, and DB_POOL_TIMEOUT_SECONDS bound how many concurrent connections the app can ever open against the database, so a traffic spike or bug can't exhaust the database's connection limit. pool_pre_ping is enabled, so dead connections are detected and discarded automatically rather than causing errors.

4. Database-Driven CORS

Which frontend origins are allowed to call this API is controlled by a database table — `allowed_origins` — not by a static value in `.env`. This means the list of permitted origins can be changed by whoever administers the database, without editing a config file or redeploying the backend.

How it works

- Every incoming request's Origin header is checked against the active rows of the `allowed_origins` table.
- To avoid querying the database on every single request, the result is cached in memory for `CORS_CACHE_TTL_SECONDS` (default 30s) and refreshed automatically once that window passes — so a change made directly in the database takes effect within one refresh cycle, no restart required.
- If a refresh attempt fails (e.g. a brief database hiccup), the middleware keeps serving the last-known-good list rather than failing open (allow everyone) or hard crashing the app.
- On a completely empty `allowed_origins` table, the app inserts two sensible defaults on first boot — `http://localhost:5173` and `http://127.0.0.1:5173` — so the bundled frontend dev server isn't locked out on a fresh database. This only happens if the table has zero rows; once anyone adds a row, this default-seeding step never runs again.

Managing allowed origins

There is a read-only endpoint — `GET /api/database/cors-origins` — and a matching “Allowed Origins” panel on the app's Database page, so you can see what's currently permitted at a glance. There is deliberately no write (POST/PUT/DELETE) endpoint for this table: this app has no login, so an unauthenticated write endpoint would let anyone grant their own origin access. Manage the table directly instead:

```
-- Add an allowed origin
INSERT INTO allowed_origins (origin, is_active, note, created_at)
VALUES ('https://app.yourcompany.com', true, 'Production frontend', now());

-- Disable one without deleting it (keeps history)
UPDATE allowed_origins SET is_active = false WHERE origin = 'https://old-domain.com';

-- Remove one entirely
DELETE FROM allowed_origins WHERE origin = 'https://old-domain.com';
```

Or, programmatically, via the same SQLAlchemy session the app itself uses (see `app/services/cors_service.py` for the ORM model and cache implementation) — useful for scripting origin management as part of a deployment pipeline.

5. Table Schema

patient_visits

Field	Description
id	Integer, primary key, auto-increment.
record_id	String(32), unique, indexed — a deterministic hash used to prevent duplicate inserts.
patient_name	String(255), indexed — the field searched (fuzzy match) from the dashboard.
visit_date	Date, indexed — nullable.

Field	Description
hospital / doctor / city	String, indexed — nullable.
diseases	JSON — a list of diagnosis strings for that visit, nullable.
claim_amount	Float — nullable.
notes	Text — free-form notes, nullable.
source_file	String(255) — which data-load batch a row came from.
created_at	DateTime — when the row was inserted, defaults to the current UTC time.

allowed_origins

Field	Description
id	Integer, primary key, auto-increment.
origin	String(500), unique, indexed — e.g. https://app.yourcompany.com (scheme + host + port, no trailing slash).
is_active	Boolean — inactive rows are kept for history but not enforced.
note	String(255), optional — a human-readable label for who/what this origin is.
created_at	DateTime — when the row was added.

6. Running with Docker Compose

docker-compose.yml provisions a PostgreSQL 16 container alongside the backend and frontend. Copy the root .env.example to .env, set a real DB_PASSWORD, then:

```
cp .env.example .env # sets DB_NAME / DB_USER / DB_PASSWORD for the db container
docker compose up --build
docker compose exec backend python -m scripts.seed_dummy_data # load demo data
```

The backend's DATABASE_URL is automatically overridden inside docker-compose.yml to point at the db service using those same credentials, so backend/.env and the root .env stay in sync without manual duplication.

7. Troubleshooting

Field	Description
“connection refused”	The database isn't running, or DB_HOST/port is wrong. For Docker Compose, confirm the db service is healthy:
“password authentication failed”	DB_USER/DB_PASSWORD don't match what the database user was created with — check for typos or unescaped
“SSL is required”	The database enforces SSL but DB_SSL_MODE is set to disable/allow. Set it to require.
Dashboard shows “No data in the connected database yet”	The connected database table is empty — run python -m scripts.seed_dummy_data.
Browser console shows a CORS error	Backend's origin isn't in the allowed_origins table (or is marked inactive). Insert/enable it, then wait up to 60
Changes to .env aren't taking effect	Settings are read once at process startup — restart the backend after editing .env.

Patient 360 Analytics Dashboard — this guide covers the database connection and CORS layer only. See README.md for full application setup, and the in-app “Database” page for live connection status and allowed origins.